CENG454 – HW3 Technical Report: Core Breach

1. Cover Page

Full Name: Zeynep Aslan

Student ID: 210444028

GitHub Repository URL: https://github.com/zyesimm/CENG454-HW3-Zeynep-Aslan-210444028

Submission Date: 17 May 2026

2. Gameplay Overview

Flower Witch: Core Breach is a 2D defensive survival game where the player controls a witch tasked with protecting the LifeBlossom at the center of the arena. Enemies continuously spawn from the outer edges of the map and move toward the core using different movement behaviors.

The gameplay loop focuses on moving around the arena, avoiding enemy contact, and casting thorn spells to eliminate enemies before they destroy the LifeBlossom. Enemy pressure gradually increases over time as the spawn rate accelerates.

The player wins by surviving until the gameplay timer reaches the target duration.

The player loses if:
-the LifeBlossom is destroyed, or
-an enemy directly touches the player.

3. Architectural Overview

The project architecture was designed around modular gameplay systems with loose coupling and interface-driven communication. The gameplay loop is separated into focused systems instead of relying on a single large manager class.

Main Systems

Player System

-Handles player movement and directional input.
-Publishes player death events.
-Communicates with the spell casting system.

Spell System

-Uses the Decorator pattern to extend spell behavior.

-Supports reusable spell implementations through the ISpell interface.
-Handles projectile spawning through the Object Pool system.

Enemy System

-Enemies use interchangeable movement strategies through the Strategy pattern.
-Enemy behavior remains independent from concrete movement implementations.
-Enemies dynamically target the LifeBlossom objective.

Core Objective System

-The LifeBlossom acts as the defendable objective.
-Publishes health and destruction events using Observer-style communication.
-UI and game state systems subscribe to these events.

Game State System

-Handles start flow, win states, lose states, and gameplay timing.
-Separates gameplay state logic from combat systems.

Object Pool System

-Reuses projectile objects to reduce runtime allocations.
-Resets pooled object state through the IPoolable interface.

UI System

-Displays health, score, start screen, and end screen information.
-Reacts to gameplay events through decoupled communication.

System Communication Summary

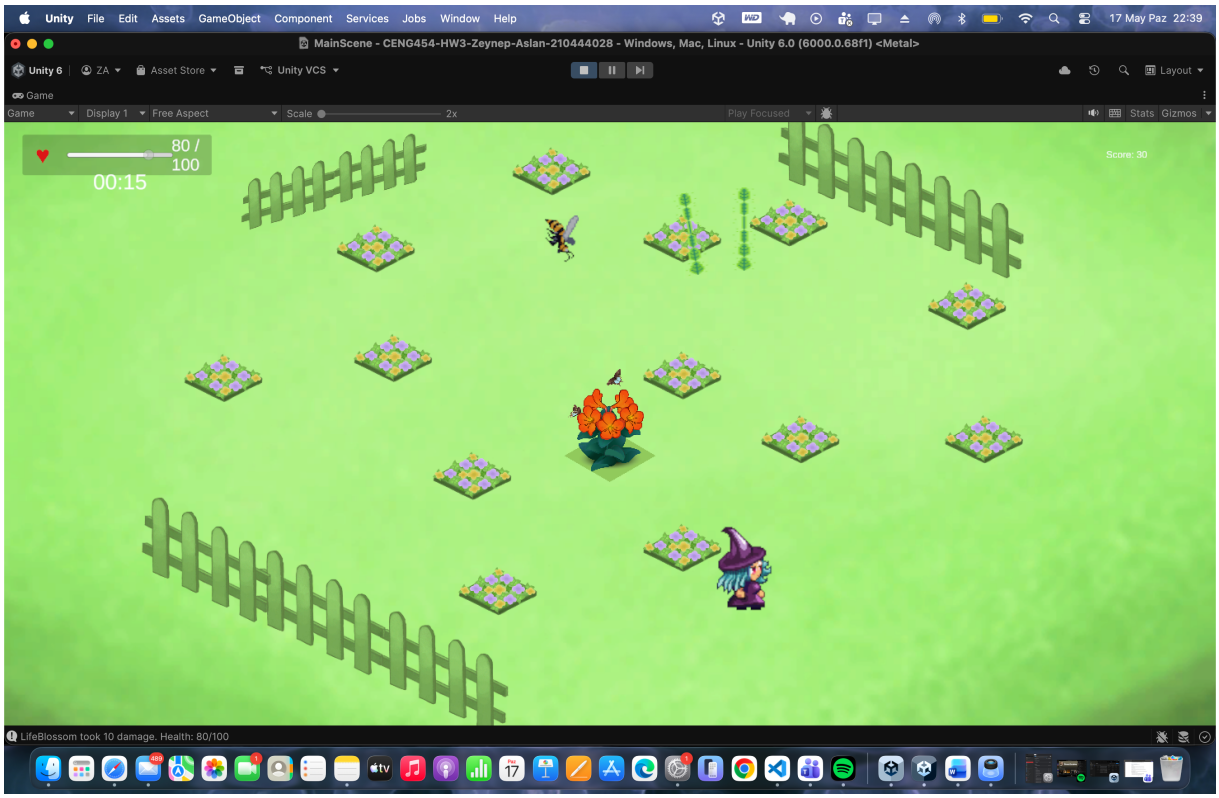
-PlayerController publishes OnPlayerKilled.
-LifeBlossom publishes OnCoreDestroyed and health update events.
-GameStateController subscribes to gameplay events.
-SpellCaster uses ISpell implementations.
-Enemy movement uses IMovementStrategy implementations.
-ThornProjectile objects are managed through ObjectPool.

4. Pattern Justification

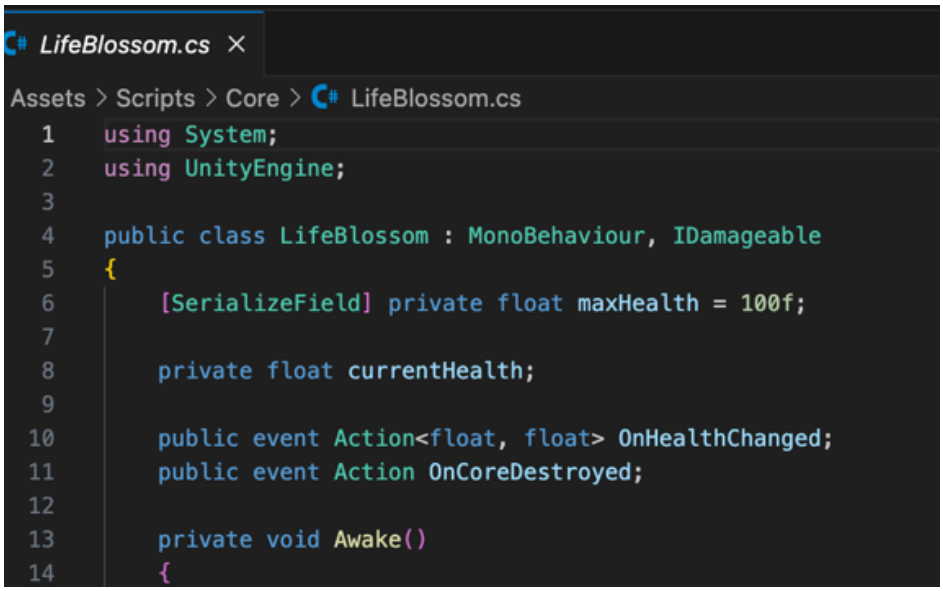
Pattern / Technique	Classes or Interfaces Involved	Problem Solved	Why This Choice Was Appropriate
Observer	LifeBlossom, GameStateController, UI systems, PlayerController	Multiple gameplay systems needed to react to core destruction and player death without direct dependencies.	Observer-style events allowed gameplay reactions to remain decoupled and modular.
Strategy	IMovementStrategy, DirectMovementStrategy, ZigzagMovementStrategy, Enemy	Enemy movement behaviors needed to be interchangeable without rewriting enemy logic.	Strategy cleanly separated movement logic from enemy ownership and allowed runtime behavior variety.
Object Pool	ObjectPool, ThornProjectile, IPoolable	Projectile spawning occurs frequently during gameplay. Repeated Instantiate/Destroy operations would reduce scalability.	Object Pool reduced runtime allocations and supported reusable projectile objects with reset behavior.
Decorator	ISpell, SpellDecorator, PoisonSpellDecorator, ThornSpell	Spell behavior needed extensibility without modifying the base spell implementation.	Decorator allowed additional spell behavior to be layered dynamically while preserving modularity.
Interfaces	IDamageable, IMovementStrategy, IPoolable, ISpell	Gameplay systems required abstraction boundaries and reusable behavior contracts.	Interfaces reduced concrete dependencies and improved extensibility across systems.

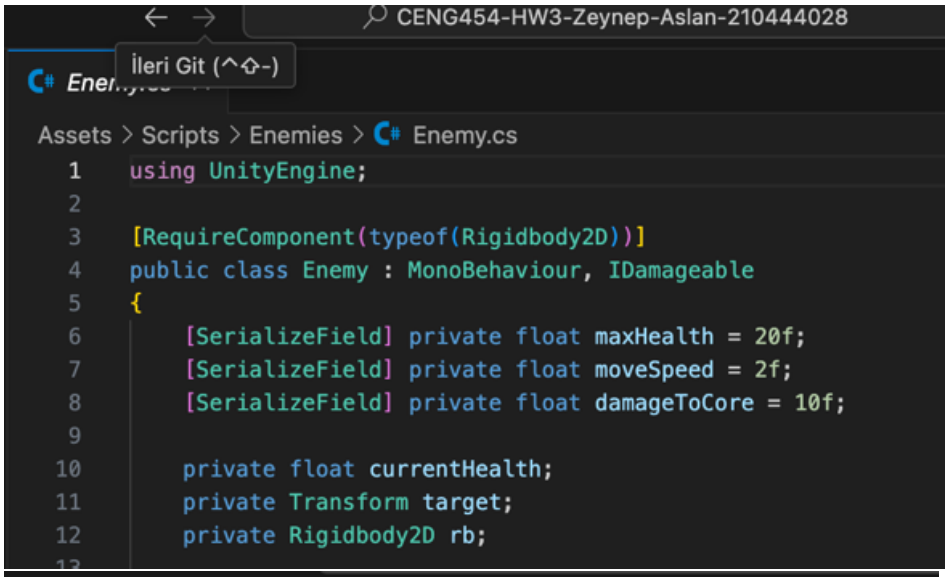
5. Screenshots

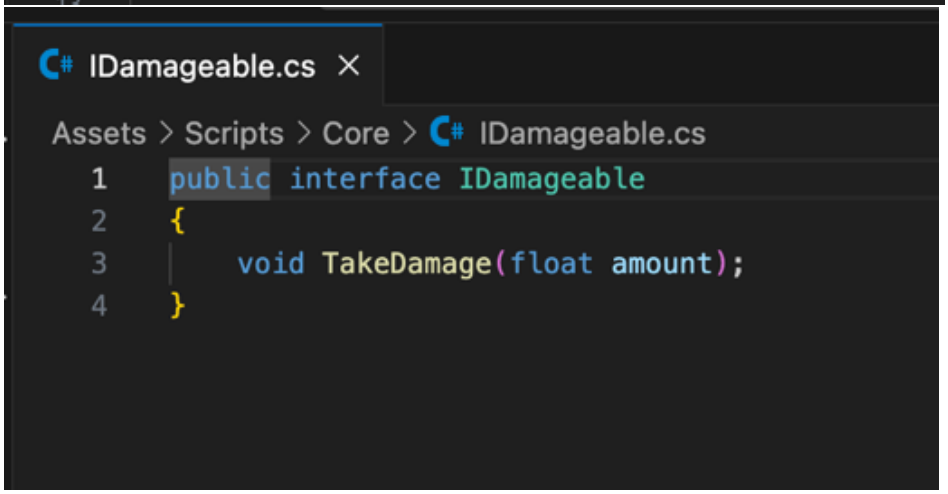
Scene and Gameview



Interfaces


LifeBlossom.cs
Assets > Scripts > Core > LifeBlossom.cs
using System;
using UnityEngine;
public class LifeBlossom : MonoBehaviour, IDamageable
{
[SerializeField] private float maxHealth = 100f;
private float currentHealth;
public event Action<float, float> OnHealthChanged;
public event Action OnCoreDestroyed;
private void Awake()
{


CENG454-HW3-Zeynep-Aslan-210444028
İleri Git (^⇧-)
Assets > Scripts > Enemies > Enemy.cs
using UnityEngine;
[RequireComponent(typeof(Rigidbody2D))]
public class Enemy : MonoBehaviour, IDamageable
{
[SerializeField] private float maxHealth = 20f;
[SerializeField] private float moveSpeed = 2f;
[SerializeField] private float damageToCore = 10f;
private float currentHealth;
private Transform target;
private Rigidbody2D rb;


IDamageable.cs
Assets > Scripts > Core > IDamageable.cs
public interface IDamageable
{
void TakeDamage(float amount);
}

Observers

GameStateController.cs

Assets > Scripts > GameFlow > GameStateController.cs

Bul (geçmiş için ⇅) Aa ab .* Sonuç yok

```
6   {
24      {
36          {
37              endPanel.SetActive(false);
38          }
39      }
40
41      private void OnEnable()
42      {
43          if (lifeBlossom != null)
44          {
45              lifeBlossom.OnCoreDestroyed += HandleCoreDestroyed;
46          }
47
48          if (playerController != null)
49          {
50              playerController.OnPlayerKilled += HandlePlayerKilled;
51          }
52      }
53
54      private void OnDisable()
55      {
56          if (lifeBlossom != null)
57          {
58              lifeBlossom.OnCoreDestroyed -= HandleCoreDestroyed;
59          }
60
61          if (playerController != null)
62          {
63              playerController.OnPlayerKilled -= HandlePlayerKilled;
64          }
65      }
```

PlayerController.cs

Assets > Scripts > Player > PlayerController.cs

Bul (geçmiş için ⇅) Aa ab .* Sonuç yok

```
1   using System;
2   using UnityEngine;
3
4   [RequireComponent(typeof(Rigidbody2D))]
5   public class PlayerController : MonoBehaviour
6   {
7       [SerializeField] private float moveSpeed = 5f;
8       [SerializeField] private SpriteRenderer spriteRenderer;
9
10      private Rigidbody2D rb;
11      private Vector2 moveInput;
12      private Vector2 facingDirection = Vector2.right;
13
14      public Vector2 FacingDirection => facingDirection;
15      public event Action OnPlayerKilled;
16
17      private void Awake()
```

```
}

private void OnTriggerEnter2D(Collider2D other)
{
    if (other.GetComponent<Enemy>() != null)
    {
        OnPlayerKilled?.Invoke();
    }
}
```

LifeBlossom.cs

Assets > Scripts > Core > LifeBlossom.cs

```
using System;
using UnityEngine;

public class LifeBlossom : MonoBehaviour, IDamageable
{
    [SerializeField] private float maxHealth = 100f;

    private float currentHealth;

    public event Action<float, float> OnHealthChanged;
    public event Action OnCoreDestroyed;

    private void Awake()
    {
        currentHealth = maxHealth;
    }

    private void Start()
    {
        OnHealthChanged?.Invoke(currentHealth, maxHealth);
    }

    public void TakeDamage(float amount)
    {

        currentHealth -= amount;
        currentHealth = Mathf.Clamp(currentHealth, 0f, maxHealth);
        Debug.Log($"LifeBlossom took {amount} damage. Health: {currentHeal
        OnHealthChanged?.Invoke(currentHealth, maxHealth);

        if (currentHealth <= 0f)
        {
            OnCoreDestroyed?.Invoke();
```

Strategy

Enemy.cs

Assets > Scripts > Enemies > Enemy.cs

```
5   {
24
25      private void FixedUpdate()
26      {
27          if (target == null) return;
28
29          Vector2 direction;
30
31          IMovementStrategy movementStrategy = GetComponent<IMovementStrategy>();
32
33          if (movementStrategy != null)
34          {
35              direction = movementStrategy.GetMovementDirection(transform.position, target.position);
36          }
37          else
38          {
39              direction = (target.position - transform.position).normalized;
40          }
41
42          rb.linearVelocity = direction * moveSpeed;
43          float angle = Mathf.Atan2(direction.y, direction.x) * Mathf.Rad2Deg;
44          transform.rotation = Quaternion.Euler(0f, 0f, angle);
45
46      }
```

DirectMovementStrategy.cs

Assets > Scripts > Strategies > DirectMovementStrategy.cs

```
1   using UnityEngine;
2
3   public class DirectMovementStrategy : MonoBehaviour, IMovementStrategy
4   {
5       public Vector2 GetMovementDirection(Vector2 currentPosition, Vector2 targetPosition
6       {
7           return (targetPosition - currentPosition).normalized;
8       }
9   }
```

ZigzagMovementStrategy.cs

Assets > Scripts > Strategies > ZigzagMovementStrategy.cs

```
using UnityEngine;

public class ZigzagMovementStrategy : MonoBehaviour, IMovementStrategy
{
    [SerializeField] private float zigzagFrequency = 4f;
    [SerializeField] private float zigzagStrength = 0.6f;

    public Vector2 GetMovementDirection(Vector2 currentPosition, Vector2 targetPosition
    {
        Vector2 directDirection = (targetPosition - currentPosition).normalized;
        Vector2 perpendicular = new Vector2(-directDirection.y, directDirection.x);

        float zigzagOffset = Mathf.Sin(Time.time * zigzagFrequency) * zigzagStrength;

        return (directDirection + perpendicular * zigzagOffset).normalized;
    }
}
```

IMovementStrategy.cs

Assets > Scripts > Strategies > IMovementStrategy.cs

```
using UnityEngine;

public interface IMovementStrategy
{
    Vector2 GetMovementDirection(Vector2 currentPosition, Vector2 targetPosition);
}
```

Decorator

SpellCaster.cs

Assets > Scripts > Combat > SpellCaster.cs

```
4    {
15       {
24           }
25
26           currentSpell = new PoisonSpellDecorator(
27               new ThornSpell(projectilePool)
28           );
29       }
```

ISpell.cs

Assets > Scripts > Spells > ISpell.cs

```
1    using UnityEngine;
2
3    public interface ISpell
4    {
5        void Cast(Vector3 origin, Vector2 direction);
6    }
```

PoisonSpellDecorator.cs

Assets > Scripts > Spells > PoisonSpellDecorator.cs

```
1   using UnityEngine;
2
3   public class PoisonSpellDecorator : SpellDecorator
4   {
5       private readonly float spreadAngle = 10f;
6
7       public PoisonSpellDecorator(ISpell wrappedSpell) : base(wrappedSpell)
8       {
9       }
10
11      public override void Cast(Vector3 origin, Vector2 direction)
12      {
13          base.Cast(origin, direction);
14
15          Vector2 spreadDirection = Quaternion.Euler(0f, 0f, spreadAngle) * direction;
16          wrappedSpell.Cast(origin, spreadDirection.normalized);
17      }
18  }
```

SpellDecorator.cs

Assets > Scripts > Spells > SpellDecorator.cs

```
1   using UnityEngine;
2
3   public abstract class SpellDecorator : ISpell
4   {
5       protected readonly ISpell wrappedSpell;
6
7       protected SpellDecorator(ISpell wrappedSpell)
8       {
9           this.wrappedSpell = wrappedSpell;
10      }
11
12      public virtual void Cast(Vector3 origin, Vector2 direction)
13      {
14          wrappedSpell.Cast(origin, direction);
15      }
16  }
```

Object pooling

ThornProjectile.cs

Assets > Scripts > Combat > ThornProjectile.cs

```
4   {
56      }
57
58      public void OnSpawned()
59      {
60          lifeTimer = lifeTime;
61      }
62
63      public void OnDespawned()
64      {
65          lifeTimer = 0f;
66          moveDirection = Vector2.zero;
67      }
```

ObjectPool.cs

Assets > Scripts > Pooling > ObjectPool.cs

```cs
using System.Collections.Generic;
using UnityEngine;

public class ObjectPool : MonoBehaviour
{
    [SerializeField] private GameObject prefab;
    [SerializeField] private int initialSize = 20;

    private readonly Queue<GameObject> pool = new Queue<GameObject>();

    private void Awake()
    {
        for (int i = 0; i < initialSize; i++)
        {
            GameObject obj = CreateNewObject();
            obj.SetActive(false);
            pool.Enqueue(obj);
        }
    }

    public GameObject GetObject(Vector3 position, Quaternion rotation)
    {
        GameObject obj = pool.Count > 0 ? pool.Dequeue() : CreateNewObject();

        obj.transform.SetPositionAndRotation(position, rotation);
        obj.SetActive(true);

        if (obj.TryGetComponent(out IPoolable poolable))
        {
            poolable.OnSpawned();
        }

        return obj;
    }

    public void ReturnObject(GameObject obj)
```

IPoolable.cs

Assets > Scripts > Interfaces > IPoolable.cs

```cs
public interface IPoolable
{
    void OnSpawned();
    void OnDespawned();
}
```

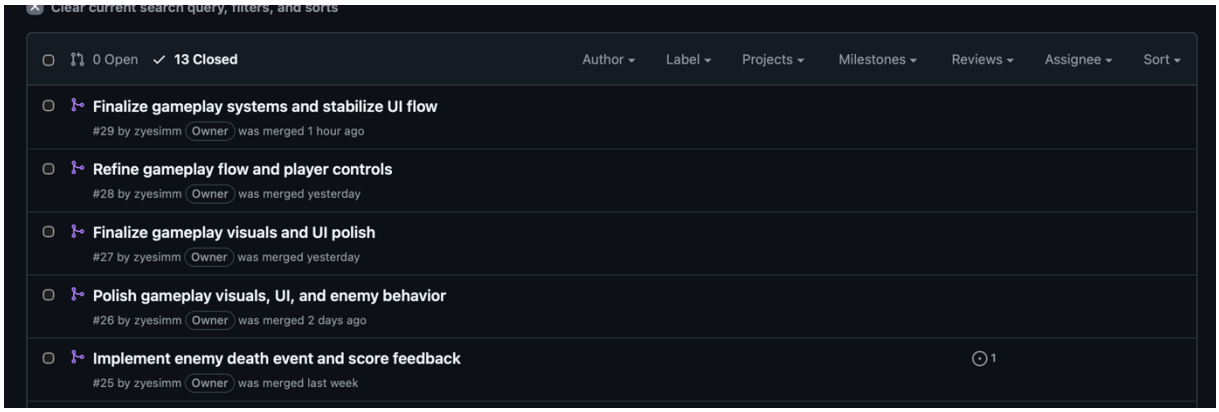
ThornProjectile.cs

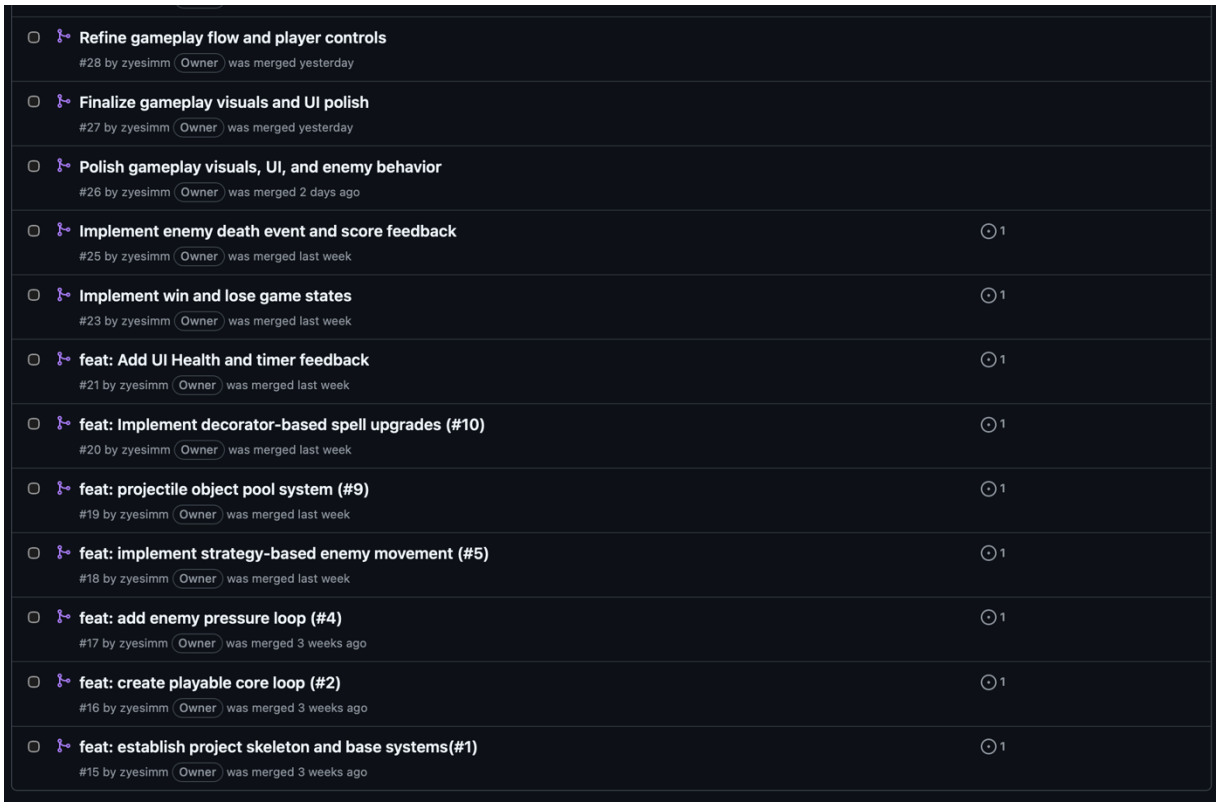
Assets > Scripts > Combat > ThornProjectile.cs

```cs
using UnityEngine;

public class ThornProjectile : MonoBehaviour, IPoolable
{
```

Repository history





6. Debugging Story

During development, I encountered a bug related to the Object Pool system used for thorn projectiles. After several projectiles were fired and returned to the pool, some reused projectiles did not behave consistently. In some cases, a projectile could keep an old movement direction or disappear earlier than expected.

I diagnosed the issue by testing repeated spell casting during Play Mode and checking how projectile state values changed between spawn and despawn. The root cause was that pooled projectiles were not being treated like newly created objects every time they were reused. Since Object Pool reuses the same GameObject instances, runtime values such as lifetime and movement direction must be manually reset.

To fix this, I introduced the IPoolable interface with OnSpawned() and OnDespawned() methods. ThornProjectile implements this interface and resets its lifetime timer when spawned, then clears its movement direction and timer when returned to the pool. This made the projectile behavior stable and predictable across multiple reuse cycles.

This bug helped me understand that Object Pool is not only about avoiding Instantiate and Destroy calls. It also requires careful lifecycle management, because every pooled object must return to a clean state before being reused.

7. Debug Problem #003

A) Visible symptom:

An enemy that has already died or returned to the object pool may still react when the core-damaged event is triggered. This can cause duplicate sounds, repeated VFX, incorrect score/logic updates, or reactions from inactive enemies.

B) Root cause:

The enemy subscribed to an event but was not unsubscribed before being returned to the pool. Since pooled objects are reused instead of destroyed, the old event subscription stays active and may stack with new subscriptions.

C) Exact Fit:

The exact fix is to unsubscribe the enemy from the core-damaged event before it is returned to the object pool. In code, this means removing the event listener with the -= operator inside the enemy's disable/despawn logic, such as OnDisable() or OnDespawned(). After that, the enemy's temporary runtime state should also be reset before reuse. This ensures that when the enemy is

spawned again, it does not carry old event subscriptions from its previous lifecycle.

D) Prevention Rule:

Every event subscription using += must have a matching unsubscription using -= in the object's lifecycle. Pooled objects should always clear event subscriptions and reset their state before returning to the pool.

8. Retrospective

This project helped me better understand how design patterns can be used in an actual gameplay system instead of only theoretical examples. During development, I realized that separating systems with interfaces and modular structures made debugging and adding new features much easier.

The most useful patterns for this project were Strategy and Observer, because they helped keep gameplay systems more organized and flexible. I also learned that Object Pool systems require careful reset logic when reusing objects.

One of the biggest challenges was managing gameplay state and object references while multiple systems interacted at the same time. Fixing these problems improved my understanding of Unity event flow and runtime behavior.

If I continued developing the project, I would add more enemy types, new spell variations, sound effects, and wave-based gameplay progression.